# Plant A Maintenance Agent

**An ontology-backed AI agent for field engineers in Japanese manufacturing**

Every answer comes with evidence you can trace on a graph.

Gemini 3.8 Flash · Agent Development Kit · Vertex AI · Cloud Run · Neo4j AuraDB

**AI Builder Cup 2026 · Manufacturing · October 2026**

Kakeru Kurosawa · Kotaro Fukuo

All data in this deck is synthetic (“Plant A”). No real company, product or person is referenced.

# Maintenance know-how that lives in four systems and in retiring veterans can become an AI agent that answers with traceable evidence, on the data a plant already has

## Problem

When a machine fails at night, the facts needed to decide are scattered across the equipment register, work orders, the parts inventory and manuals, and the veteran who connects them is off duty or retiring. Document search cannot connect them.

## Solution

We model the plant as an ontology (equipment, components, failure modes, symptoms, work orders, procedures, parts, people) in a knowledge graph. A Gemini agent built with ADK walks it through read-only tools and cites work-order IDs and manual pages.

## Proof

Deployed on Cloud Run with Neo4j AuraDB. In rehearsal, 45 of 45 reference answers were correct across diagnosis, cross-line comparison, parts and people, aggregation and “no record” questions. ADK hallucination score 1.0; median answer about 20 seconds.

## Impact

Shorter diagnosis on every unplanned stop, cross-line patterns that registers alone hide, and know-how that survives retirements. The same pipeline runs on a plant's own registers, work orders and manuals, in Japanese or English, in four weeks.

# Field engineers lose hours on every unplanned stop because the facts needed to decide live in four places, and in one veteran's head

### Equipment register & BOM

- Models, install years and component structure in a CMMS or Excel
- Finding “the same model on another line” means searching across registers

### Work orders

- “What happened, when, what we did” in a CMMS, daily reports or paper
- Similar past cases are found from memory, if at all

### Manuals

- Vendor PDFs with judgement thresholds and procedures
- Someone has to remember which page

### Veteran experience

- Which cause to suspect from a symptom; what to check on a repeat failure
- Not on the night shift; lost at retirement

**Plant A (our synthetic demo plant)** Three press procedures (slide-guide, clutch-brake and flywheel-bearing replacement) can only be performed by one technician who holds the press-operation certificate and retires at the end of FY2026. The register, the work orders and the skills ledger each know one piece of this; nobody sees it until the three are connected.

# Document search (RAG alone) cannot answer the questions that matter most on the floor: cross-line similarity, counts, and “who can do this”

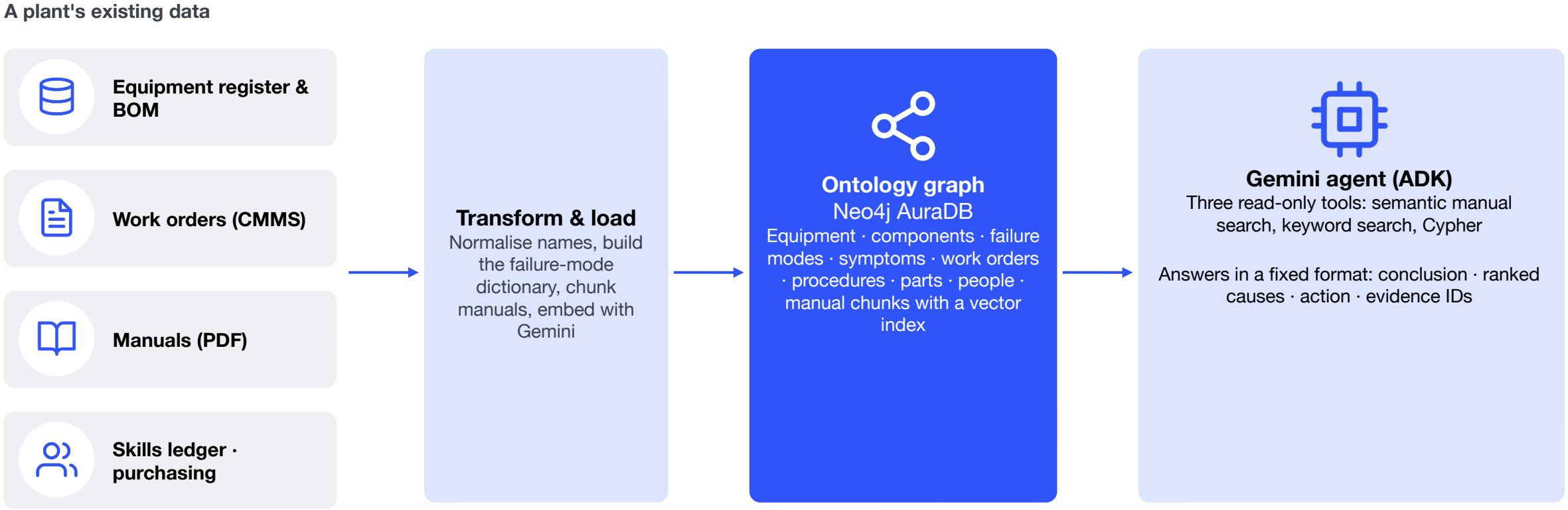
## Document search only

- Retrieves text chunks that look similar. The P-301 manual and P-301 reports are found
- Reports from the same model on another line never say “P-301”, so they are missed
- Cannot count: “which machines had the most bearing-related stops this year?”
- Evidence is a text fragment; a person must work out which machine and which report
- May infer a part number or lead time from context and state it as fact

## Knowledge graph + agent (ours)

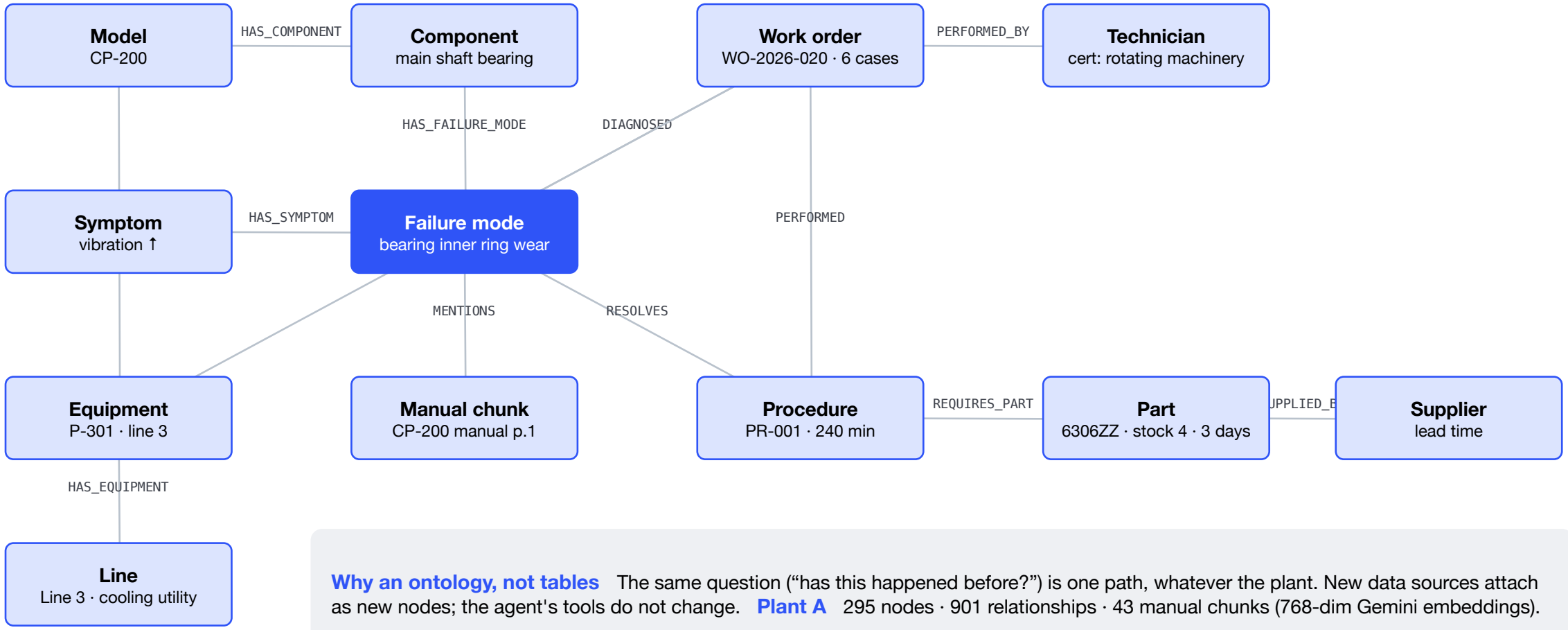
- Stores equipment, components, failure modes and work orders as things and relationships
- Crosses lines through the model node: P-301 → CP-200 ← P-101
- Aggregates by failure-mode category and date in one query
- Evidence is work-order IDs, procedure IDs and manual pages, drawn as a sub-graph
- Answers “no record” when the graph has nothing; never invents an ID

# Vector search over the manuals finds where to start; the knowledge graph supplies the ranked answer, the next action and the evidence



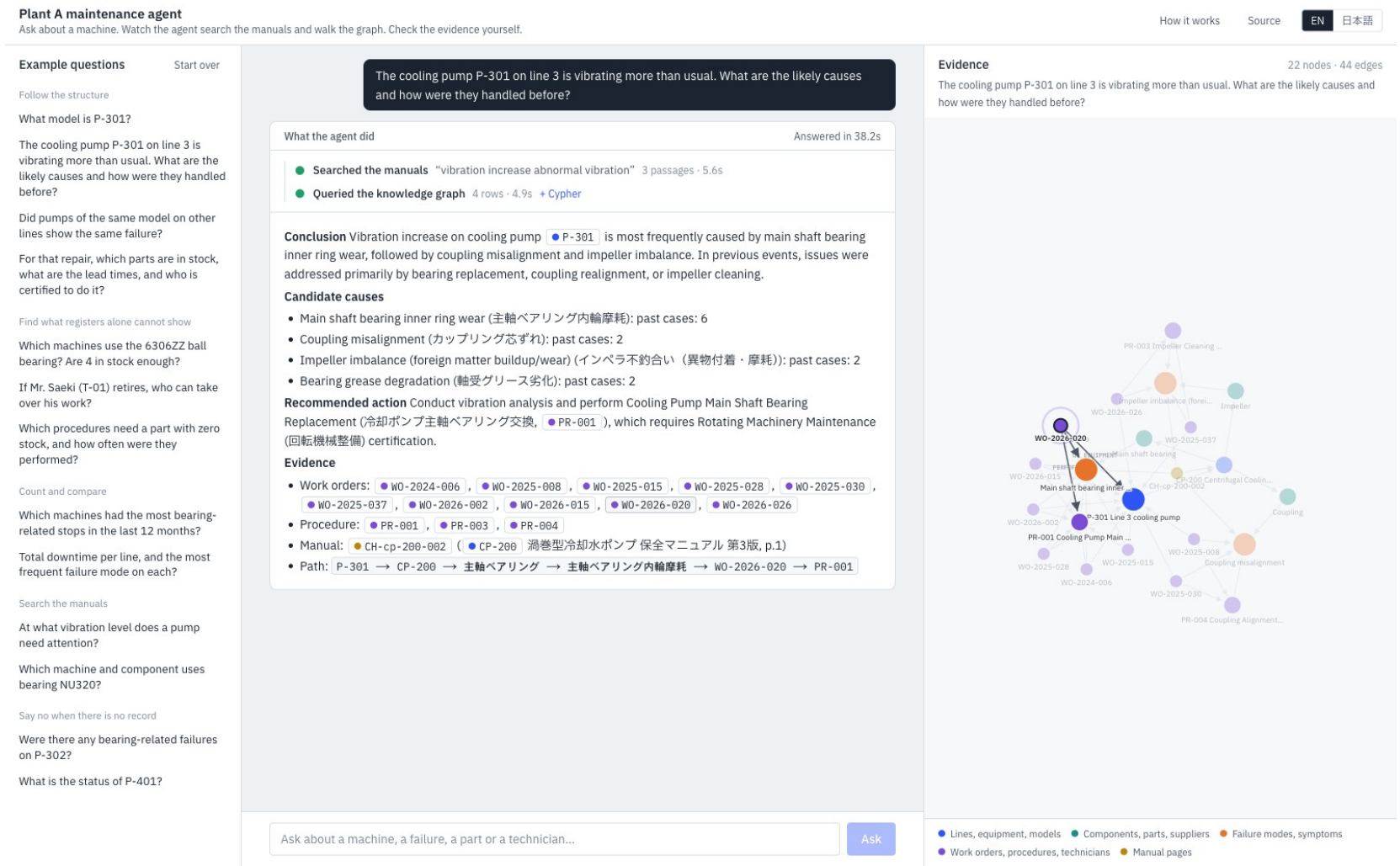

**Three mechanisms** 1 Vector search picks the entry point (a manual passage linked to failure modes). 2 The graph walk delivers the answer: causes ranked by history, procedure, parts in stock, certified people. 3 Every ID in the answer is a node the user can inspect on screen.

# Twelve node types and seventeen relationships, with the failure mode as the hub, cover diagnosis, history, the procedure, parts and people in one walk




**Why an ontology, not tables** The same question (“has this happened before?”) is one path, whatever the plant. New data sources attach as new nodes; the agent's tools do not change. **Plant A** 295 nodes · 901 relationships · 43 manual chunks (768-dim Gemini embeddings).

# In the live app every cited ID is a clickable node: the engineer sees what the agent did, reads the answer, and checks the evidence on the graph



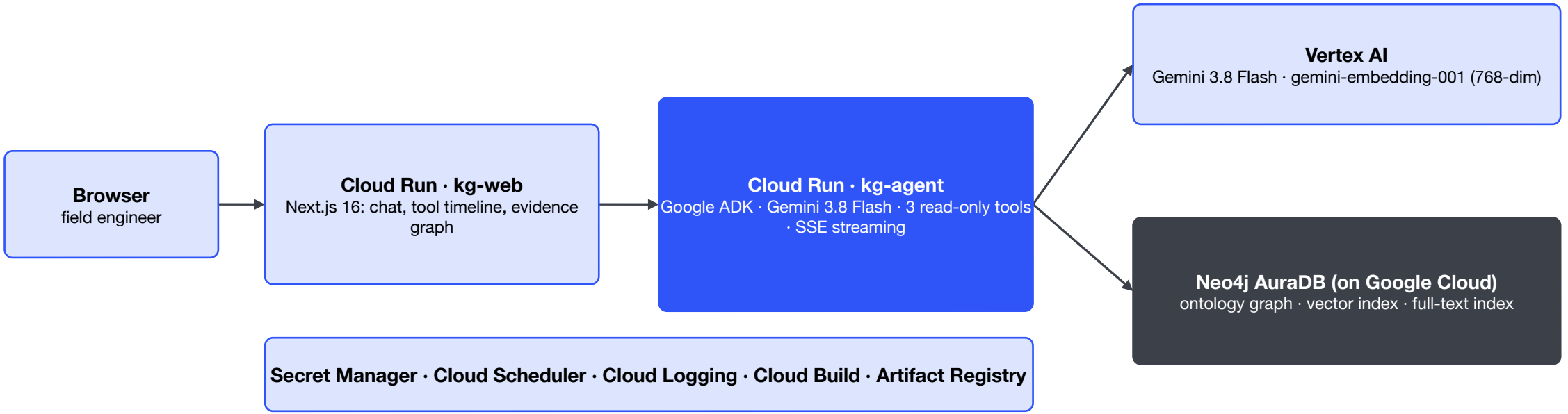

**1 What the agent did**
Semantic manual search, then one Cypher query. Each step shows the search text or the Cypher, row counts and seconds.

**2 Answer in a fixed format**
Conclusion · candidate causes with past-case counts · recommended action with the required certification · evidence IDs.

**3 Evidence graph**
Shortest paths between the cited nodes. Clicking WO-2026-020 in the answer highlights it on the graph.

**4 Bilingual over Japanese records**
Ask in English or Japanese; the graph values stay in Japanese with English names, IDs never change.

# An ADK agent with three read-only tools, Gemini for reasoning and embeddings, two Cloud Run services, and guardrails that reject writes and invented IDs




### Read-only by construction

Write clauses are rejected before execution; transactions open in READ mode; 20-second query timeout. Covered by unit tests.

### Evidence is mandatory

Fixed answer format with an Evidence section. IDs become chips and are resolved against the graph, so an unknown ID cannot appear as a node.

### Private agent, public UI

kg-agent accepts only ID tokens from the web service account and Cloud Scheduler. Neo4j credentials come from Secret Manager.

### Evaluated, not assumed

ADK eval set with LLM-judged response match and hallucination metrics, plus a rehearsal harness with fact checkers (next slide).

# Rehearsal and ADK evaluation: 45 of 45 reference answers correct, hallucination score 1.0, median answer about 20 seconds with two to three tool calls

**45 / 45**
reference answers correct
(9 cases × 5 runs, fact-checked)

**1.0**
ADK hallucinations_v1
(7 / 7 eval cases pass)

**≈ 20 s**
median answer time
2–3 tool calls

**0**
invented work-order IDs,
parts or people

| Case | Question | Correct | Median (min–max) | Tool calls |
|---|---|---|---|---|
| Q1 | P-301 vibrating: causes and past fixes (JA / EN) | 10 / 10 | 23 s (18–32) | 2.4 |
| Q2 | Same model on other lines (follow-up) | 5 / 5 | 32 s (22–42) | 2.0 |
| Q3 | Parts in stock, lead times, certified technicians | 5 / 5 | 24 s (22–30) | 2.2 |
| Q4 | Most bearing-related stops in the last 12 months | 5 / 5 | 26 s (20–37) | 2.6 |
| B2 | Who can take over when T-01 retires | 5 / 5 | 35 s (19–44) | 3.2 |
| D2 | Which machine uses bearing NU320 (keyword) | 5 / 5 | 20 s (18–44) | 3.0 |
| E1 / E2 | Bearing failures on P-302 · unknown machine → “no record” | 10 / 10 | 19–23 s | 2.4–3.4 |

Rehearsal on the deployed AuraDB graph with gemini-3.8-flash, 2026-10-06 (agent/rehearse.py, log in docs/eval). ADK eval: final_response_match_v2 = 1.00 on all 7 cases, hallucinations_v1 = 1.00 on 6 cases and 0.86 on the three-turn conversation (judge gemini-3.5-flash, 3 samples).

# For a plant the value is hours of downtime avoided on every incident, patterns that registers hide, and know-how that survives retirements

**80**

unplanned stops in two years at Plant A

**935 min**

bearing-related downtime on one pump in 12 months

**1**

technician holding the only press certification, retiring FY2026

**≈ 20 s**

from question to ranked causes, parts and people

## Faster diagnosis on every stop

- Plant A: 80 unplanned stops in two years, 9 bearing-related in the last 12 months alone (P-301: 3 stops, 935 minutes).
- The agent returns ranked causes, the procedure, parts and people in about 20 seconds instead of a search across four systems.
- Illustrative: one hour saved per incident × 40 incidents a year × ¥500k per hour of line stop (assumption) ≈ ¥20M a year per plant.

## Patterns registers hide

- Same-model failures across lines (P-101 ×4, P-201 ×1) surface in one answer.
- A part shared by 8 machines with stock for two replacements; procedures that depend on a zero-stock part.
- Bearing-related downtime concentrated on the CP-200 pumps: a maintenance-plan decision, not a repair.

## Know-how that survives

- One technician holds the only press certification and retires in FY2026; the graph shows which procedures have no successor.
- Work orders become searchable history; manual pages become cited evidence.
- New engineers and night shifts get the veteran's path, with the IDs to verify it.

Cost figures are illustrative assumptions for a mid-size plant, to be replaced with a customer plant's own downtime cost and incident counts in the four-week pilot.

# What is new: answers grounded in an ontology with graph evidence, bilingual access to Japanese floor documents, and an agent that is read-only and says “no record”

## Ontology-grounded GraphRAG

- Vector search only chooses where to start; the content of the answer comes from graph structure, so every claim maps to a node and a path.
- Cross-line, aggregation and “who can do it” questions that document RAG cannot answer become one Cypher query.

## Built for Japanese plants

- Registers, work orders and manuals stay in Japanese; the master data carries Gemini-generated English names.
- An engineer asks in either language and gets the same IDs. JAPAC plants with mixed teams get one assistant.

## Honest and safe by design

- The agent cannot write to the graph and rejects write clauses before execution.
- When there is no record it says so and lists only the IDs it checked. Verified in rehearsal and ADK eval.

**Why Gemini + ADK made this possible in days**

Gemini 3.8 Flash writes correct Cypher against a schema it is shown once and keeps a fixed answer format across languages; gemini-embedding-001 indexes Japanese manuals without a local model; ADK gives the tool loop, SSE streaming to the UI, Cloud Run deployment and an evaluation harness. The whole build, from synthetic plant to deployed app, took one week.

# Four weeks from a plant's own registers to a working assistant; next: sensor time series, approved write-back, and a mobile UI for the floor

**Week 1**
Data inventory, ontology fit, load the equipment register, parts and people

**Week 2**
Work orders: normalise names, build the failure-mode dictionary, load history

**Week 3**
Manuals: chunk, embed, link to components and failure modes; define reference questions

**Week 4**
Agent prompts, evaluation with floor engineers, evidence graph, hand-over

## After the first four weeks

**Sensor time series**
Attach vibration and temperature trends to equipment nodes; predict the failure mode and propose the procedure before the stop.

**Approved write-back**
Draft the work order from the conversation; a person approves before anything is written to the graph.

**Mobile UI for the floor**
Voice and camera input on a phone, offline manual pages, the same evidence graph.

## Team

**Kakeru Kurosawa**
**Product, domain & engineering lead**
Built the ontology, the agent, the graph pipeline and the web app. COO at AIdeaLab; previously PwC consulting and new-business development for edge AI at Sony, working with manufacturing customers.

**Kotaro Fukuo**
**Software engineering**
Software engineer with 8+ years in mobile and backend (React Native, TypeScript, Ruby on Rails, AWS); owns the field-facing mobile UI on the roadmap. B.E. Mechanical Engineering Informatics, Meiji University.

# Try it, read the code, ask it a question.

Live app kg-web-7ikzkb2evq-an.a.run.app
Source github.com/kurorooooo/kg-maintenance-agent

Kakeru Kurosawa · Kotaro Fukuo